# Practical – 13

## Practical Title: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem Definition: Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

## A. Theory

1. Do PHP endpoints return valid JSON responses?
Yes. The PHP endpoint uses json_encode() to convert PHP data into JSON format. It returns structured responses containing success status, messages, and student records.

2. Are GET and POST requests handled properly?
Yes. GET requests are used to retrieve student records, while POST requests handle adding, updating, and deleting student records. Both request methods are processed through api.php.

3. Are loading, success, and error states shown on the frontend?
Yes. JavaScript displays a loading message while fetching records, success messages after successful operations, and error messages when an operation fails or server communication is unsuccessful.

4. Is input validation performed before database operations?
Yes. Server-side validation checks that all required fields are filled and that the email address is valid. Prepared statements are also used to execute database queries securely.

## B. Implementation/ Output/Result:

The Student Management System was developed using PHP, MySQL, HTML, and JavaScript Fetch API. A MySQL database named practical13_db was created to store student details such as name, email, and course. PHP database connectivity was established using MySQLi and prepared statements. A PHP JSON API was implemented to handle GET and POST requests for performing CRUD operations, including adding, displaying, updating, and deleting student records. JavaScript Fetch API was integrated to communicate with the PHP backend asynchronously, allowing all operations to be performed without reloading the entire webpage. Server-side validation was implemented to check required fields and verify email formats before storing or modifying records. The frontend also displays loading messages, success messages, and error messages to provide feedback during different operations.

Result: The AJAX-based Student Management System was successfully implemented and tested, demonstrating asynchronous CRUD operations, JSON communication, database connectivity, and input validation.

## C. Evidence of Work (Screenshots):

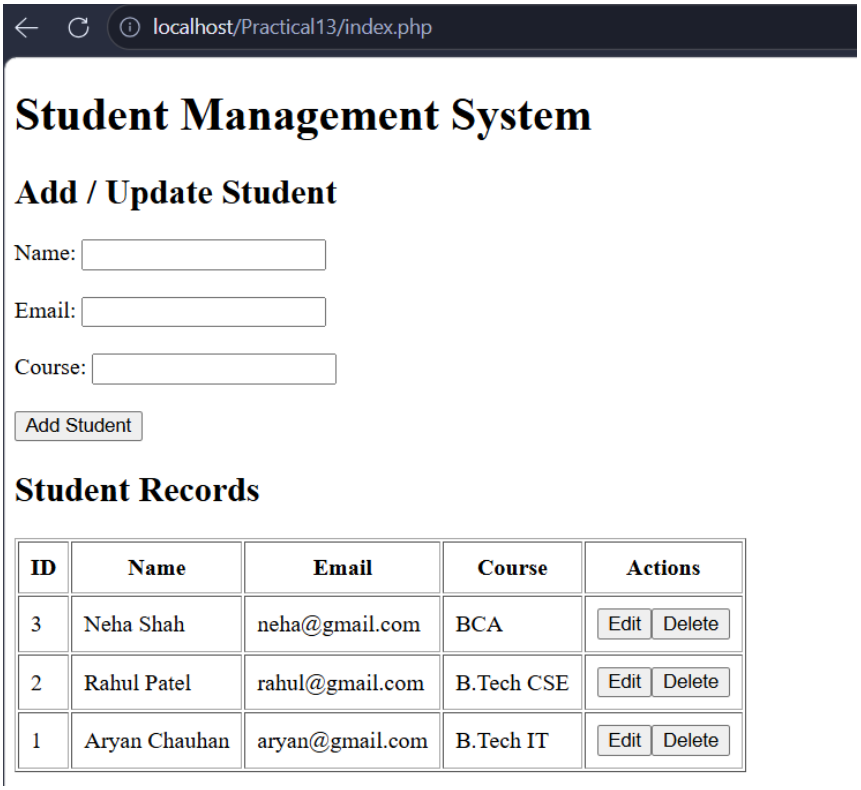



**Fig1: Student Management Dashboard**




**Fig2: Add Student**

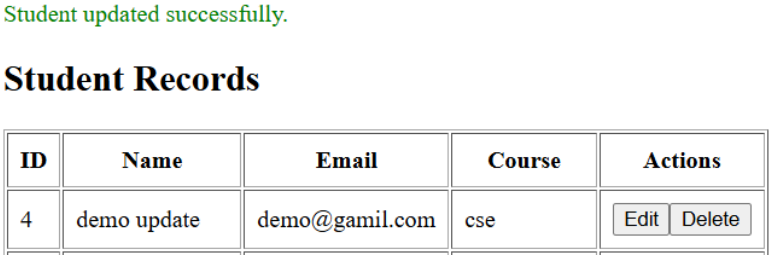



**Fig3: Update Student**

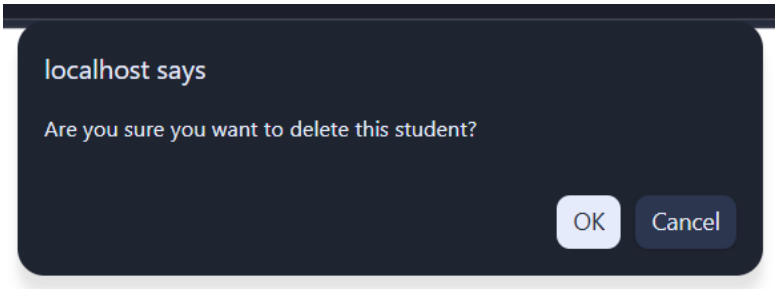



**Fig4: Delete Student**

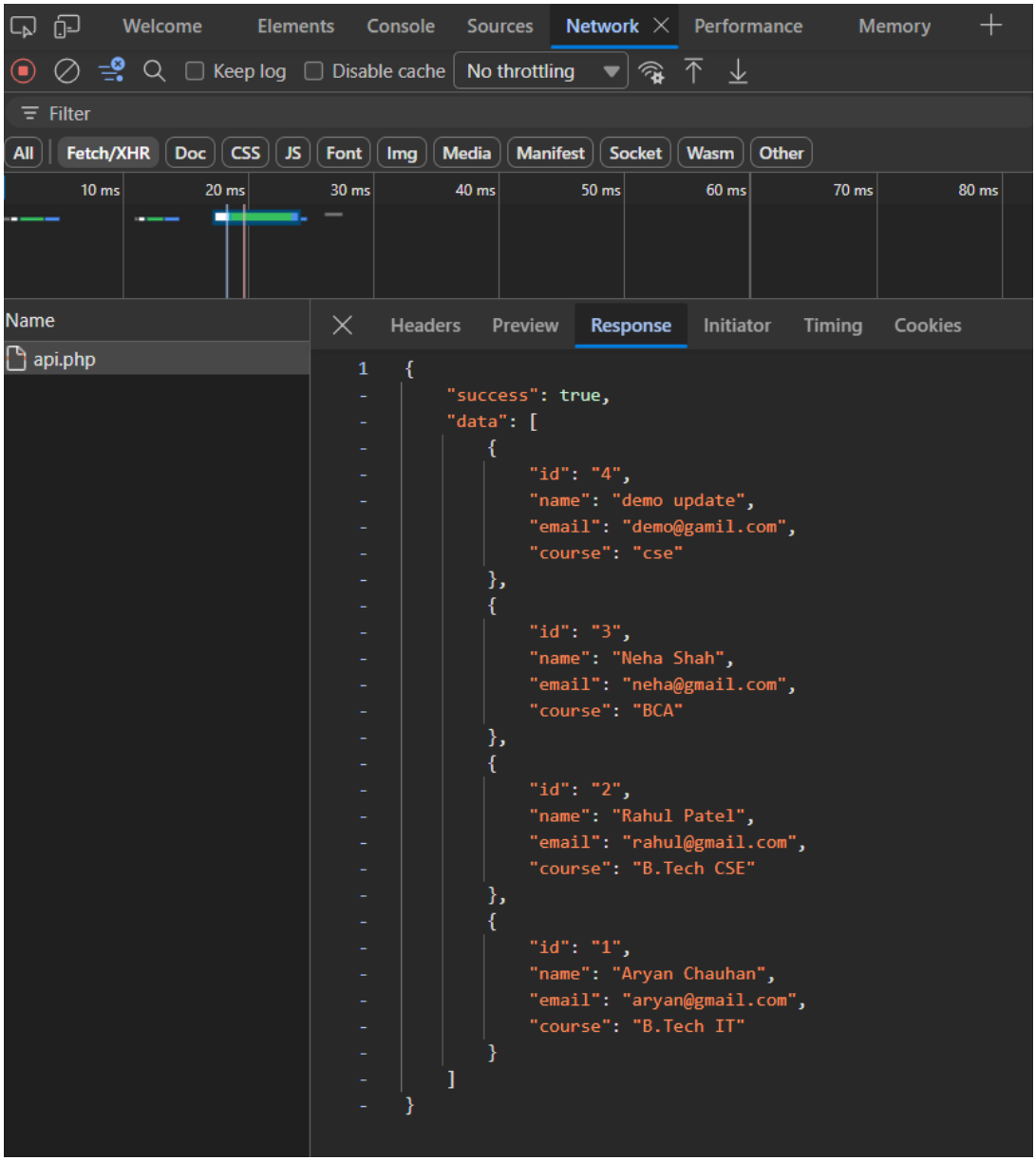



**Fig5: JSON API Response**

## D. GitHub Repository Link:

aryan150806-wq/StudentHubPortal: Web Development Framework P

## E. Challenges Faced:

1. Establishing proper communication between the PHP backend and MySQL database.
2. Generating valid JSON responses and handling GET and POST requests correctly.
3. Implementing Fetch API to perform CRUD operations without reloading the webpage.
4. Handling input validation and displaying appropriate loading, success, and error messages.

## F. Learning Outcomes:

The PHP JSON API and AJAX-based Student Management System was successfully implemented using PHP, MySQL, HTML, and JavaScript. The application performs add, view, update, and delete operations without full-page reloads. Fetch API enables asynchronous communication between the frontend and backend through JSON responses. Server-side validation ensures that invalid data is not stored in the database. Overall, this practical helped in understanding API development and asynchronous web application functionality.

## Faculty Remarks: